Relatório – Tarefa 2: Números e Display

Instituição	Universidade de Fortaleza (UNIFOR)
Curso	Engenharia da Computação
Disciplina	Projeto de Circuitos Digitais
Professor	Reinaldo Imbiriba
Aluno	Rafael Marins Florenzano
Matrícula	2214571
Atividade	Tarefa 2 – Números e Display (AV2)

1. Introdução

Este relatório apresenta o projeto, em VHDL, de seis circuitos combinacionais que convertem números binários em dígitos decimais e somam dígitos BCD, todos voltados para a placa DE0 (FPGA Cyclone III EP3C16F484C6). As seis partes seguem o roteiro “Exercício 02 – Números e Display” da disciplina.

Para cada parte são apresentadas duas soluções: a solução de referência fornecida pelo professor, escrita em soma de produtos (SOP) ou com comandos sequenciais, e uma solução alternativa proposta pelo aluno com outra técnica de descrição. As soluções alternativas foram simuladas no ModelSim-Altera e as formas de onda capturadas no GTKWave são apresentadas na seção de resultados.

A placa DE0 possui apenas os displays HEX0 a HEX3 e os LEDs verdes LEDG0 a LEDG9. Por isso, as saídas que o roteiro direciona para HEX5, HEX4 e LEDR foram remapeadas para HEX3, HEX2 e LEDG nas soluções alternativas.

2. Objetivos

Objetivo geral: projetar, descrever em VHDL e verificar por simulação circuitos combinacionais de conversão numérica e de soma BCD, comparando diferentes formas de descrição do mesmo circuito.

Objetivos específicos:

1. Deduzir as expressões booleanas (SOP) de um decodificador BCD para display de 7 segmentos (Parte 1).
2. Construir um conversor binário de 4 bits para dois dígitos decimais com comparador, circuito A e multiplexador (Parte 2).
3. Implementar um somador ripple-carry de 4 bits a partir de somadores completos (Parte 3).
4. Projetar um somador BCD de um dígito com detecção de entradas inválidas (Parte 4).
5. Descrever o mesmo somador BCD de forma comportamental e comparar os circuitos gerados no RTL Viewer (Parte 5).
6. Converter um número binário de 6 bits em dois dígitos BCD (Parte 6).

7. Para cada parte, propor uma solução alternativa com comandos concorrentes ou sequenciais diferentes dos da solução de referência e validá-la por simulação.

3. Revisão bibliográfica

3.1 Display de 7 segmentos

O display de 7 segmentos forma dígitos acendendo os segmentos 0 a 6 (topo, superior direito, inferior direito, base, inferior esquerdo, superior esquerdo e centro). Na DE0 os displays são de **anodo comum**, ou seja, um segmento acende com nível lógico **0**. Um decodificador BCD–7 segmentos é um circuito combinacional de 4 entradas e 7 saídas, uma função booleana por segmento.

Dígito	B3 B2 B1 B0	H(0..6)
0	0000	0000001
1	0001	1001111
2	0010	0010010
3	0011	0000110
4	0100	1001100
5	0101	0100100
6	0110	0100000
7	0111	0001111
8	1000	0000000
9	1001	0000100
10–15	1010–1111	não importa (X)

3.2 Soma de produtos e mapas de Karnaugh

Toda função booleana pode ser escrita como **soma de produtos (SOP)**: a OR de termos AND, um para cada linha da tabela-verdade em que a saída vale 1 (mintermos). O mapa de Karnaugh agrupa mintermos adjacentes em potências de 2 para eliminar variáveis e obter a expressão mínima. Combinações de entrada que nunca ocorrem, como 1010–1111 em BCD, são marcadas como **don't care** e podem ser incluídas nos grupos quando isso os aumenta.

3.3 Código BCD

No código **BCD** (*Binary-Coded Decimal*), cada dígito decimal é representado por 4 bits, de 0000 a 1001. As combinações 1010 a 1111 são inválidas. Assim, o número 19 em BCD é `0001 1001`, e não o binário puro `10011`.

3.4 Somador completo e somador ripple-carry

O **somador completo** soma três bits (a, b e o carry de entrada ci) e produz a soma s e o carry de saída co:

$$s = a \oplus b \oplus ci \qquad co = a \cdot b + ci \cdot (a \oplus b)$$

No **somador ripple-carry** de n bits, n somadores completos são encadeados e o carry de saída de cada estágio alimenta o carry de entrada do seguinte. O circuito é simples, mas o atraso cresce com n, pois o carry precisa atravessar todos os estágios.

3.5 Somador BCD

A soma de dois dígitos BCD mais o carry de entrada vai de 0 a 19. Quando a soma binária passa de 9, o resultado precisa ser corrigido. Há duas formas equivalentes de fazer isso:

- **Subtrair 10** do resultado e gerar dezena 1, como no pseudocódigo do roteiro (Parte 5).
- **Somar 6** (`0110`) ao resultado de 4 bits e descartar o carry. Somar 6 em módulo 16 equivale a subtrair 10, e é o método clássico dos somadores BCD.

A condição de correção é k = cout + S3·S2 + S3·S1, verdadeira quando houve carry ou quando a soma de 4 bits vale de 10 a 15.

3.6 Conversão binário–BCD: double dabble

O algoritmo **double dabble** (*shift-and-add-3*) converte binário em BCD deslocando os bits, do mais significativo ao menos significativo, para dentro dos dígitos BCD. Antes de cada deslocamento, todo dígito maior ou igual a 5 recebe +3. Assim, quando ele é multiplicado por 2 no deslocamento, o valor passa de 9 e o excesso vai como carry para o próximo dígito decimal. Em hardware combinacional, o algoritmo vira um arranjo de células “add3”.

3.7 Comandos concorrentes e sequenciais em VHDL

Em VHDL, os comandos **concorrentes** ficam diretamente no corpo da arquitetura e são avaliados em paralelo: atribuição simples (<= com expressões booleanas), atribuição condicional (`WHEN ... ELSE`), atribuição selecionada (`WITH ... SELECT`), instanciação de componentes e `FOR ... GENERATE`. Os comandos **sequenciais** (`IF`, `CASE`, `FOR LOOP`) só podem aparecer dentro de um `PROCESS`, que é executado quando muda um sinal da sua lista de sensibilidade. Ambos os estilos descrevem hardware; a diferença está no nível de abstração e no quanto o sintetizador precisa inferir.

4. Metodologia

4.1 Ferramentas e organização

Ferramenta	Uso
Quartus II 13.1 Web Edition	Edição, análise e síntese do VHDL; RTL Viewer
ModelSim-Altera 10.1d	Compilação (`vcom -93`) e simulação funcional
GTKWave 3.3	Visualização das formas de onda (arquivos VCD) e captura das telas
Placa DE0 (Cyclone III EP3C16F484C6)	Plataforma de destino: chaves SW, LEDs LEDG e displays HEX0–HEX3

Todas as partes ficam em um único projeto Quartus, `tarefa2`. Cada parte tem dois arquivos: `partN.vhd`, com a solução de referência, e `partN_alt.vhd`, com a solução alternativa. As entidades da alternativa levam o sufixo `_alt`, para que as duas versões convivam no mesmo projeto, e a parte a compilar é escolhida pelo *Top-Level Entity*. Componentes já validados são reaproveitados entre partes: o decodificador `bcd7seg_alt` da Parte 1 e o somador completo `fa_alt` da Parte 3.

Na transcrição das soluções de referência foram feitos três ajustes, sem mudar a lógica:

- Entidades repetidas entre as partes (`bcd7seg`, `fa`, `mux2to1_4bit`) são definidas uma única vez, e as de nome igual com portas diferentes foram renomeadas (`part2_conv`, `bcd7seg_case`).
- O VHDL-93 não aceita expressões como `"000" & z` diretamente no `PORT MAP` (erro *vcom-1436* no ModelSim). Nesses casos, a expressão foi atribuída antes a um sinal intermediário `d1`.
- As portas HEX5 e HEX4 das soluções de referência foram mantidas, mas não existem na DE0.

Simulação. A entidade de topo é simulada no ModelSim com estímulos de contagem binária nas chaves: o bit SW(0) alterna a cada 50 ns e cada bit seguinte com o dobro do período. Assim, todas as 2^n combinações de entrada são percorridas, uma a cada 50 ns. A simulação grava um arquivo VCD, aberto no GTKWave com os sinais de interesse. Além da inspeção visual, cada VCD foi conferido por script contra o valor esperado em todas as combinações.

4.2 Parte 1 – Decodificador BCD para 7 segmentos

Especificação. SW7–4 é exibido em HEX1 e SW3–0 em HEX0, e LEDG7–0 repete as chaves. Um mesmo componente `bcd7seg` é instanciado duas vezes, uma por dígito.

Solução de referência (SOP). Como os segmentos são ativos em 0, cada saída H(i) vale **1 nos dígitos em que o segmento fica apagado**. Para cada segmento, os mintermos foram tirados da tabela da seção 3.1 (dígitos 0–9), e as entradas 10–15 foram tratadas como don't care no mapa de Karnaugh. A notação B3′ indica NOT B3.

Segmento	Apagado nos dígitos	Grupos no mapa de Karnaugh	Expressão SOP
H(0) topo	1, 4	mintermos isolados m1 e m4	B3′B2′B1′B0 + B3′B2B1′B0′
H(1) sup. dir.	5, 6	m5 + X13; m6 + X14 (eliminam B3)	B2B1′B0 + B2B1B0′
H(2) inf. dir.	2	m2 + X10 (elimina B3)	B2′B1B0′
H(3) base	1, 4, 7	m1, m4 e m7 isolados	B3′B2′B1′B0 + B3′B2B1′B0′ + B3′B2B1B0
H(4) inf. esq.	1, 3, 4, 5, 7, 9	(1,5,9,X13); (1,3,5,7); (4,5)	B1′B0 + B3′B0 + B3′B2B1′
H(5) sup. esq.	1, 2, 3, 7	(3,7,X11,X15); (2,3,X10,X11); (1,3)	B1B0 + B2′B1 + B3′B2′B0
H(6) centro	0, 1, 7	(7,X15); (0,1)	B2B1B0 + B3′B2′B1′

O uso dos don't cares reduz os termos de H(1), H(2), H(4) e H(5). Em H(0) e H(3) os mintermos ficaram sem agrupamento, e a expressão foi mantida na forma canônica. As sete equações foram conferidas contra a tabela para os dígitos 0–9.

Solução alternativa (WITH SELECT). Em vez de uma equação por segmento, o decodificador é descrito pela própria tabela-verdade, com uma atribuição selecionada concorrente que escolhe o vetor H inteiro a partir de B. A cláusula `WHEN OTHERS` trata as entradas 1010–1111, que apagam o display (`1111111`). Esse estilo elimina a etapa manual de simplificação, que passa a ser feita pelo sintetizador, e deixa o código legível como tabela.

Código-fonte: `part1_alt.vhd`

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part1_alt IS

   PORT (
      SW      :   IN STD_LOGIC_VECTOR(7 DOWNTO 0);
      LEDG   :   OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
      HEX1, HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END part1_alt;

ARCHITECTURE Structure OF part1_alt IS
   COMPONENT bcd7seg_alt
      PORT (
         B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         H : OUT STD_LOGIC_VECTOR(0 TO 6)
      );
   END COMPONENT;
BEGIN
   LEDG <= SW;

   digit1 : bcd7seg_alt PORT MAP (SW(7 DOWNTO 4), HEX1);
   digit0 : bcd7seg_alt PORT MAP (SW(3 DOWNTO 0), HEX0);
END Structure;

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY bcd7seg_alt IS
   PORT (
      B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
      H : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END bcd7seg_alt;

ARCHITECTURE rtl OF bcd7seg_alt IS
BEGIN
   -- Segmentos ativos em 0: H(0) = topo, H(6) = centro.
   WITH B SELECT
      H <= "0000001" WHEN "0000", -- 0
           "1001111" WHEN "0001", -- 1
           "0010010" WHEN "0010", -- 2
           "0000110" WHEN "0011", -- 3
           "1001100" WHEN "0100", -- 4
           "0100100" WHEN "0101", -- 5
           "0100000" WHEN "0110", -- 6
           "0001111" WHEN "0111", -- 7
           "0000000" WHEN "1000", -- 8
           "0000100" WHEN "1001", -- 9
           "1111111" WHEN OTHERS;
END rtl;
```

4.3 Parte 2 – Conversor binário (4 bits) para decimal

Especificação. V = SW3–0 (0 a 15) é exibido como dois dígitos decimais, d1 em HEX1 e d0 em HEX0. A estrutura da Figura 1 do roteiro tem quatro blocos: um comparador z (V > 9), um circuito A (que gera V − 10), um multiplexador 2:1 de 4 bits (d0 = V se z = 0, A se z = 1) e d1 = `000z`.

Solução de referência (SOP).

- **Comparador:** z = 1 para V = 10 a 15. No mapa de Karnaugh, os grupos (12, 13, 14, 15) e (10, 11, 14, 15) dão **z = V3V2 + V3V1**.
- **Circuito A:** só importa para V > 9, quando V3 = 1. Por isso, V3 sai da tabela e as entradas 0–9 viram don’t care.

V	V2 V1 V0	A2 A1 A0

10	0 1 0	0 0 0
11	0 1 1	0 0 1
12	1 0 0	0 1 0
13	1 0 1	0 1 1
14	1 1 0	1 0 0
15	1 1 1	1 0 1

Da tabela: **A3 = 0**, **A2 = V2V1**, **A1 = V2V1′** e **A0 = V1V0 + V2V0**. O multiplexador é escrito bit a bit como **M(i) = s′X(i) + sY(i)**.

Solução alternativa (WITH SELECT e WHEN ELSE). Os mesmos blocos da Figura 1 são mantidos, mas sem nenhuma equação:

- o comparador e o circuito A são atribuições selecionadas (`WITH V SELECT`) que listam diretamente as linhas da tabela-verdade;
- o multiplexador vira uma atribuição condicional de uma linha (`M <= X WHEN s = '0' ELSE Y`), que vale para os 4 bits de uma vez;
- os displays usam o `bcd7seg_alt` da Parte 1.

Código-fonte: `part2_alt.vhd`

```
-- Conversor binario (4 bits) para decimal de 2 digitos - versao alternativa.
-- Mesmo diagrama de blocos da Figura 1 (comparador, circuito A, mux 2:1 e
-- decodificadores 7-seg), mas descrito com comandos concorrentes WITH SELECT
-- e WHEN ELSE no lugar das equacoes SOP.

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part2_alt IS
   PORT (
      SW         : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
      HEX1, HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END part2_alt;

ARCHITECTURE Structure OF part2_alt IS
   COMPONENT mux2to1_4bit_alt
      PORT (
         X, Y : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         s    : IN STD_LOGIC;
         M    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
      );
   END COMPONENT;
   -- bcd7seg_alt definido em part1_alt.vhd
   COMPONENT bcd7seg_alt
      PORT (
         B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         H : OUT STD_LOGIC_VECTOR(0 TO 6)
      );
   END COMPONENT;
   SIGNAL V, A, M : STD_LOGIC_VECTOR(3 DOWNTO 0);
   SIGNAL z       : STD_LOGIC;
   SIGNAL d1      : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
   V <= SW;

   -- Comparador: z = 1 para V > 9, tirado direto da tabela verdade.
   WITH V SELECT
      z <= '1' WHEN "1010" | "1011" | "1100" | "1101" | "1110" | "1111",
           '0' WHEN OTHERS;

   -- Circuito A: para V > 9 entrega o digito das unidades (V - 10).
   -- Para V <= 9 a saida nao importa, pois o mux seleciona V.
   WITH V SELECT
```

```
      A <= "0000" WHEN "1010", -- 10 -> 0
           "0001" WHEN "1011", -- 11 -> 1
           "0010" WHEN "1100", -- 12 -> 2
           "0011" WHEN "1101", -- 13 -> 3
           "0100" WHEN "1110", -- 14 -> 4
           "0101" WHEN "1111", -- 15 -> 5
           "0000" WHEN OTHERS;

   U0 : mux2to1_4bit_alt PORT MAP (V, A, z, M);

   U1 : bcd7seg_alt PORT MAP (M, HEX0);
   d1 <= "000" & z;
   U2 : bcd7seg_alt PORT MAP (d1, HEX1);
END Structure;

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Multiplexador 2:1 de 4 bits com atribuicao condicional.
ENTITY mux2to1_4bit_alt IS
   PORT (
      X, Y : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
      s    : IN STD_LOGIC;
      M    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
   );
END mux2to1_4bit_alt;

ARCHITECTURE rtl OF mux2to1_4bit_alt IS
BEGIN
   M <= X WHEN s = '0' ELSE Y;
END rtl;
```

4.4 Parte 3 – Somador ripple-carry de 4 bits

Especificação. A = SW7–4, B = SW3–0 e cin = SW8. A saída cout & S (0 a 31) aparece em LEDG4–0. O roteiro exige uma entidade para o somador completo e um nível superior com quatro instâncias dela.

Solução de referência. O somador completo `fa` usa as equações da seção 3.4, reescritas com um sinal intermediário a⊕b: **s = (a⊕b)⊕ci** e **co = (a⊕b)′·b + (a⊕b)·ci**. Quando a = b, o carry é o próprio b; quando a ≠ b, o carry de entrada se propaga. As quatro instâncias são escritas à mão (`bit0` a `bit3`), ligadas pelo vetor de carries C(4..0).

Solução alternativa (tabela-verdade e FOR GENERATE). Duas técnicas diferentes, mantendo a exigência de quatro instâncias:

- **`fa_alt`:** os três bits de entrada são concatenados (`a & b & ci`), e um `WITH SELECT` devolve o par `co s` para cada uma das 8 linhas da tabela-verdade da Figura 2c. O somador sai da tabela, sem nenhuma equação.
- **Topo:** as quatro instâncias são geradas por `FOR i IN 0 TO 3 GENERATE`, e `C(i+1)` liga o carry de saída do bit i ao carry de entrada do bit i+1. Para um somador de n bits, basta mudar o limite do laço.

Código-fonte: `part3_alt.vhd`

```
-- Somador ripple-carry de 4 bits - versao alternativa.
-- O somador completo e descrito pela sua tabela verdade (WITH SELECT) e as
-- quatro instancias sao criadas com FOR GENERATE.
-- Entradas: SW7-4 = A, SW3-0 = B, SW8 = cin
-- Saidas:   LEDG4 = cout, LEDG3-0 = S

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part3_alt IS
   PORT (
```

```
      SW   : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
      LEDG : OUT STD_LOGIC_VECTOR(9 DOWNTO 0)
   );
END part3_alt;

ARCHITECTURE Structure OF part3_alt IS
   COMPONENT fa_alt
      PORT (
         a, b, ci : IN STD_LOGIC;
         s, co    : OUT STD_LOGIC
      );
   END COMPONENT;
   SIGNAL A, B, S : STD_LOGIC_VECTOR(3 DOWNTO 0);
   SIGNAL C       : STD_LOGIC_VECTOR(4 DOWNTO 0);
BEGIN
   A <= SW(7 DOWNTO 4);
   B <= SW(3 DOWNTO 0);
   C(0) <= SW(8);

   -- o carry de saida do bit i alimenta o carry de entrada do bit i+1
   adders : FOR i IN 0 TO 3 GENERATE
      fa_i : fa_alt PORT MAP (A(i), B(i), C(i), S(i), C(i + 1));
   END GENERATE;

   LEDG(4 DOWNTO 0) <= C(4) & S;
   LEDG(9 DOWNTO 5) <= "00000";
END Structure;

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Somador completo a partir da tabela verdade da Figura 2c: co s = a + b + ci.
ENTITY fa_alt IS
   PORT (
      a, b, ci : IN STD_LOGIC;
      s, co    : OUT STD_LOGIC
   );
END fa_alt;

ARCHITECTURE rtl OF fa_alt IS
   SIGNAL entrada : STD_LOGIC_VECTOR(2 DOWNTO 0);
   SIGNAL saida   : STD_LOGIC_VECTOR(1 DOWNTO 0);
BEGIN
   entrada <= a & b & ci;

   --  a b ci | co s
   WITH entrada SELECT
      saida <= "00" WHEN "000",
               "01" WHEN "001",
               "01" WHEN "010",
               "10" WHEN "011",
               "01" WHEN "100",
               "10" WHEN "101",
               "10" WHEN "110",
               "11" WHEN "111",
               "00" WHEN OTHERS;

   co <= saida(1);
   s  <= saida(0);
END rtl;
```

4.5 Parte 4 – Somador BCD de um dígito

Especificação. X = SW7–4, Y = SW3–0 e cin = SW8. A soma S1S0 = X + Y + cin (0 a 19) aparece em HEX1 e HEX0, e a soma binária de 5 bits em LEDG4–0. LEDG9 acende quando X ou Y é maior que 9. Na DE0, X e Y são exibidos em HEX3 e HEX2, já que não existem HEX5 e HEX4.

Solução de referência. Reaproveita os circuitos das Partes 2 e 3:

1. O ripple-carry da Parte 3 calcula S (4 bits) e C4.

2. O conversor da Parte 2 (`part2_conv`) transforma S em z e M, o que resolve as somas de 0 a 15.
3. Para as somas de 16 a 19 (C4 = 1), S vale 0 a 3 e o dígito correto é 6 a 9. Uma pequena lógica gera **B = (M1, M1′, M1′, M0)**, que leva 0→6, 1→7, 2→8 e 3→9. Um multiplexador controlado por C4 escolhe entre M e B.
4. A dezena é **S1 = z + C4**, e o erro é **LEDG9 = X3X2 + X3X1 + Y3Y2 + Y3Y1**, o mesmo comparador “> 9” da Parte 2 aplicado a X e Y.

Solução alternativa (correção +6). Usa o método clássico de somador BCD (seção 3.5), dispensando o conversor da Parte 2 e o mux:

1. **Primeiro somador:** `adder4_alt` (ripple-carry com `FOR GENERATE` sobre o `fa_alt` da Parte 3) calcula S e cout = X + Y + cin.
2. **Detecção:** `k = 1 WHEN cout = '1' OR S > 9`, ou seja, a soma passou de 9.
3. **Correção:** `corr = "0110" WHEN k = '1' ELSE "0000"`. Um segundo `adder4_alt` faz S + corr, e o carry desse segundo somador é descartado. O resultado é o dígito S0, e **S1 = k**.
4. **Erro:** LEDG9 usa os comparadores `unsigned(X) > 9 OR unsigned(Y) > 9`, do pacote `numeric_std`.

Código-fonte: `part4_alt.vhd`

```
-- Somador BCD de 1 digito S1 S0 = X + Y + cin - versao alternativa.
-- Usa o metodo classico de correcao "soma 6": quando a soma binaria passa de 9,
-- um segundo somador de 4 bits soma 0110 para ajustar o digito das unidades,
-- e o digito das dezenas vira 1.
-- Entradas: SW7-4 = X, SW3-0 = Y, SW8 = cin
-- Saidas:   X em HEX3, Y em HEX2 (a DE0 nao tem HEX5/HEX4),
--           S1 S0 em HEX1 HEX0,
--           LEDG4-0 = soma binaria (cout & S), LEDG9 = erro (X ou Y > 9)

LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY part4_alt IS
   PORT (
      SW                    : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
      LEDG                  : OUT STD_LOGIC_VECTOR(9 DOWNTO 0);
      HEX3, HEX2, HEX1, HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END part4_alt;

ARCHITECTURE Structure OF part4_alt IS
   COMPONENT adder4_alt
      PORT (
         a, b : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         cin  : IN STD_LOGIC;
         s    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
         cout : OUT STD_LOGIC
      );
   END COMPONENT;
   -- bcd7seg_alt definido em part1_alt.vhd
   COMPONENT bcd7seg_alt
      PORT (
         B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         H : OUT STD_LOGIC_VECTOR(0 TO 6)
      );
   END COMPONENT;
   SIGNAL X, Y, S, corr, S0, d1 : STD_LOGIC_VECTOR(3 DOWNTO 0);
   SIGNAL cin, cout, k, cout2  : STD_LOGIC;
BEGIN
   X <= SW(7 DOWNTO 4);
```

```
   Y <= SW(3 DOWNTO 0);
   cin <= SW(8);

   -- 1o somador: soma binaria X + Y + cin (0 a 19 para entradas validas)
   add1 : adder4_alt PORT MAP (X, Y, cin, S, cout);
   LEDG(4 DOWNTO 0) <= cout & S;

   -- k = 1 quando a soma binaria passa de 9: houve carry (16-19) ou S > 9 (10-15)
   k <= '1' WHEN cout = '1' OR unsigned(S) > 9 ELSE '0';

   -- 2o somador: S + 6 corrige o digito das unidades; o carry dele e descartado
   corr <= "0110" WHEN k = '1' ELSE "0000";
   add2 : adder4_alt PORT MAP (S, corr, '0', S0, cout2);

   d1 <= "000" & k;
   H1 : bcd7seg_alt PORT MAP (d1, HEX1);
   H0 : bcd7seg_alt PORT MAP (S0, HEX0);

   -- entradas nos displays e deteccao de digito BCD invalido
   H3 : bcd7seg_alt PORT MAP (X, HEX3);
   H2 : bcd7seg_alt PORT MAP (Y, HEX2);
   LEDG(9) <= '1' WHEN unsigned(X) > 9 OR unsigned(Y) > 9 ELSE '0';
   LEDG(8 DOWNTO 5) <= "0000";
END Structure;

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Somador ripple-carry de 4 bits com o fa_alt da Parte 3 (part3_alt.vhd).
ENTITY adder4_alt IS
   PORT (
      a, b : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
      cin  : IN STD_LOGIC;
      s    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
      cout : OUT STD_LOGIC
   );
END adder4_alt;

ARCHITECTURE Structure OF adder4_alt IS
   COMPONENT fa_alt
      PORT (
         a, b, ci : IN STD_LOGIC;
         s, co    : OUT STD_LOGIC
      );
   END COMPONENT;
   SIGNAL C : STD_LOGIC_VECTOR(4 DOWNTO 0);
BEGIN
   C(0) <= cin;
   adders : FOR i IN 0 TO 3 GENERATE
      fa_i : fa_alt PORT MAP (a(i), b(i), C(i), s(i), C(i + 1));
   END GENERATE;
   cout <= C(4);
END Structure;
```

4.6 Parte 5 – Somador BCD comportamental

Especificação. É o mesmo somador BCD da Parte 4 (A = SW7–4, B = SW3–0, cin = SW8), agora descrito em alto nível a partir do pseudocódigo do roteiro. O circuito gerado deve ser comparado com o da Parte 4 no RTL Viewer.

Solução de referência (PROCESS e IF). Segue o pseudocódigo:

1. T0 = A + B + cin, com o operador + do pacote `std_logic_unsigned`.
2. Um PROCESS sensível a T0 testa `IF T0 > 9`. Se sim, Z0 = 10 e C1 = 1; se não, Z0 = 0 e C1 = 0.
3. Fora do processo, S0 = T0 − Z0 e S1 = C1.

Como o roteiro pede um comando sequencial, a solução de referência já usa `IF`. Por isso, a alternativa foi feita sem `PROCESS` e sem `IF`.

Solução alternativa (aritmética concorrente). A correção é eliminada:

- a soma é convertida em inteiro: `T = TO_INTEGER(A + B + cin)`, com `numeric_std`;
- os dígitos saem direto da divisão inteira e do resto por 10: **S1 = T / 10** e **S0 = T MOD 10**;
- não há comparação com 9 nem multiplexador.

Como a divisão e o resto valem para qualquer valor de 0 a 31, esta versão também mostra o resultado correto para entradas fora do BCD (por exemplo, 15 + 15 + 1 = 31). A versão de referência só é correta até 19.

Código-fonte: `part5_alt.vhd`

```
-- Somador BCD de 1 digito S1 S0 = A + B + cin - versao alternativa.
-- Totalmente concorrente e aritmetico: sem PROCESS, sem IF e sem comparacao
-- com 9. A soma vira um inteiro e os digitos saem direto de T / 10 e T MOD 10.
-- Entradas: SW7-4 = A, SW3-0 = B, SW8 = cin
-- Saidas:   A em HEX3, B em HEX2 (a DE0 nao tem HEX5/HEX4), S1 S0 em HEX1 HEX0

LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY part5_alt IS
   PORT (
      SW                    : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
      HEX3, HEX2, HEX1, HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END part5_alt;

ARCHITECTURE Behavior OF part5_alt IS
   -- bcd7seg_alt definido em part1_alt.vhd
   COMPONENT bcd7seg_alt
      PORT (
         B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         H : OUT STD_LOGIC_VECTOR(0 TO 6)
      );
   END COMPONENT;
   SIGNAL A, B, S1, S0 : STD_LOGIC_VECTOR(3 DOWNTO 0);
   SIGNAL ci           : UNSIGNED(0 DOWNTO 0);
   SIGNAL T            : INTEGER RANGE 0 TO 31;
BEGIN
   A <= SW(7 DOWNTO 4);
   B <= SW(3 DOWNTO 0);
   ci(0) <= SW(8);

   -- soma de 5 bits (0 a 31) convertida para inteiro
   T <= TO_INTEGER(('0' & UNSIGNED(A)) + UNSIGNED(B) + ci);

   -- dezenas e unidades por divisao inteira e resto
   S1 <= STD_LOGIC_VECTOR(TO_UNSIGNED(T / 10, 4));
   S0 <= STD_LOGIC_VECTOR(TO_UNSIGNED(T MOD 10, 4));

   H3 : bcd7seg_alt PORT MAP (A, HEX3);
   H2 : bcd7seg_alt PORT MAP (B, HEX2);
   H1 : bcd7seg_alt PORT MAP (S1, HEX1);
   H0 : bcd7seg_alt PORT MAP (S0, HEX0);
END Behavior;
```

4.7 Parte 6 – Conversor binário (6 bits) para BCD

Especificação. O número binário SW5–0 (0 a 63) é exibido em decimal: dezenas em HEX1 e unidades em HEX0, com HEX3 e HEX2 apagados.

Solução de referência (PROCESS e IF/ELSIF). Um `PROCESS` compara a entrada com as faixas < 10, < 20, …, < 60. Na faixa encontrada, a dezena recebe uma constante (0 a 6) e a unidade recebe a entrada menos o limite inferior da faixa. O sintetizador gera uma cadeia de comparadores e subtratores, com multiplexadores em prioridade.

Solução alternativa (double dabble estrutural). Usa o algoritmo da seção 3.6, sem nenhum `IF`. A célula `add3` é um `WITH SELECT` que soma 3 quando a entrada vale de 5 a 9 e repassa o valor nos demais casos.

No arranjo clássico para 8 bits há sete células. Com 6 bits de entrada, os bits mais altos são zero, e quatro dessas células recebem sempre valores menores que 5, virando simples fios. Restam três células encadeadas:

1. **U3** recebe `0 b5 b4 b3`.
2. **U5** recebe os 3 bits baixos de U3 deslocados, mais b2.
3. **U7** recebe os 3 bits baixos de U5 deslocados, mais b1.
4. O bit mais alto de cada célula é o que transborda para as dezenas: **dezena = 0 & c3(3) & c5(3) & c7(3)**. O último deslocamento traz b0 para as unidades: **unidade = c7(2..0) & b0**.

A ligação foi validada antes, em script, para os 64 valores de entrada.

Código-fonte: `part6_alt.vhd`

```
-- Conversor binario de 6 bits para BCD de 2 digitos - versao alternativa.
-- Algoritmo double dabble (shift-and-add-3): os bits entram da esquerda para a
-- direita e, antes de cada deslocamento, todo digito BCD >= 5 recebe +3.
-- Para 6 bits so tres celulas add3 sao necessarias; nas demais posicoes do
-- arranjo a entrada e sempre < 5 e a celula vira um fio.
-- Entradas: SW5-0 = numero binario (0 a 63)
-- Saidas:   dezenas em HEX1, unidades em HEX0, HEX3 e HEX2 apagados

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part6_alt IS
   PORT (
      SW                     : IN STD_LOGIC_VECTOR(5 DOWNTO 0);
      HEX3, HEX2, HEX1, HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END part6_alt;

ARCHITECTURE Structure OF part6_alt IS
   COMPONENT add3
      PORT (
         x : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         y : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
      );
   END COMPONENT;
   -- bcd7seg_alt definido em part1_alt.vhd
   COMPONENT bcd7seg_alt
      PORT (
         B : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         H : OUT STD_LOGIC_VECTOR(0 TO 6)
      );
   END COMPONENT;
   SIGNAL e3, e5, e7    : STD_LOGIC_VECTOR(3 DOWNTO 0);   -- entradas das celulas
   SIGNAL c3, c5, c7    : STD_LOGIC_VECTOR(3 DOWNTO 0);   -- saidas das celulas
   SIGNAL dezena, unid  : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
   -- 1a celula: os tres bits mais significativos (b5 b4 b3)
   e3 <= '0' & SW(5 DOWNTO 3);
   U3 : add3 PORT MAP (e3, c3);

   -- 2a celula: resultado deslocado + b2
```

```
   e5 <= c3(2 DOWNTO 0) & SW(2);
   U5 : add3 PORT MAP (e5, c5);

   -- 3a celula: resultado deslocado + b1
   e7 <= c5(2 DOWNTO 0) & SW(1);
   U7 : add3 PORT MAP (e7, c7);

   -- o bit que "transborda" de cada celula forma o digito das dezenas;
   -- o ultimo deslocamento traz b0 para as unidades
   dezena <= '0' & c3(3) & c5(3) & c7(3);
   unid   <= c7(2 DOWNTO 0) & SW(0);

   H1 : bcd7seg_alt PORT MAP (dezena, HEX1);
   H0 : bcd7seg_alt PORT MAP (unid, HEX0);

   HEX3 <= "1111111";
   HEX2 <= "1111111";
END Structure;

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Celula add3: y = x + 3 se x >= 5, senao y = x.
-- Entradas 10-15 nunca ocorrem no double dabble.
ENTITY add3 IS
   PORT (
      x : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
      y : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
   );
END add3;

ARCHITECTURE rtl OF add3 IS
BEGIN
   WITH x SELECT
      y <= "1000" WHEN "0101", -- 5 -> 8
           "1001" WHEN "0110", -- 6 -> 9
           "1010" WHEN "0111", -- 7 -> 10
           "1011" WHEN "1000", -- 8 -> 11
           "1100" WHEN "1001", -- 9 -> 12
           x      WHEN OTHERS;
END rtl;
```

5. Resultados

As seis soluções alternativas produziram a saída esperada em todas as combinações de entrada simuladas. Os arquivos VCD de cada simulação foram conferidos por script, combinação a combinação, contra o valor calculado.

Parte	Entradas	Combinações verificadas	Erros
1	SW7–0	256	0
2	SW3–0	16	0
3	SW8–0	512	0
4	SW8–0	512 (200 com X e Y válidos)	0
5	SW8–0	512	0
6	SW5–0	64	0

Nas capturas, cada combinação dura 50 ns. As chaves e os valores intermediários aparecem em decimal ou hexadecimal, e os displays em binário, no formato H(0..6) com segmentos ativos em 0.

5.1 Parte 1



Figura 1 – Parte 1: SW de 00 a 0F (0 a 800 ns)

De 0 a 800 ns, SW vai de `00` a `0F`. HEX0 percorre os dez padrões de 0 (`0000001`) a 9 (`0000100`) e fica apagado (`1111111`) de A a F. HEX1 mostra 0 e LEDG acompanha SW.

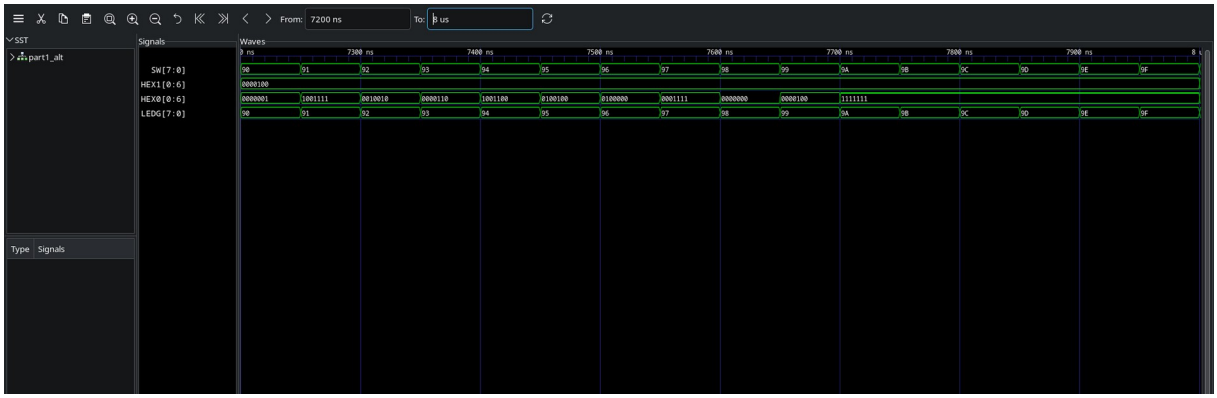


Figura 2 – Parte 1: SW de 90 a 9F (7200 a 8000 ns)

De 7200 a 8000 ns, o nibble alto vale 9: HEX1 mostra `0000100` e HEX0 repete o ciclo de 0 a 9, seguido do apagado.

5.2 Parte 2

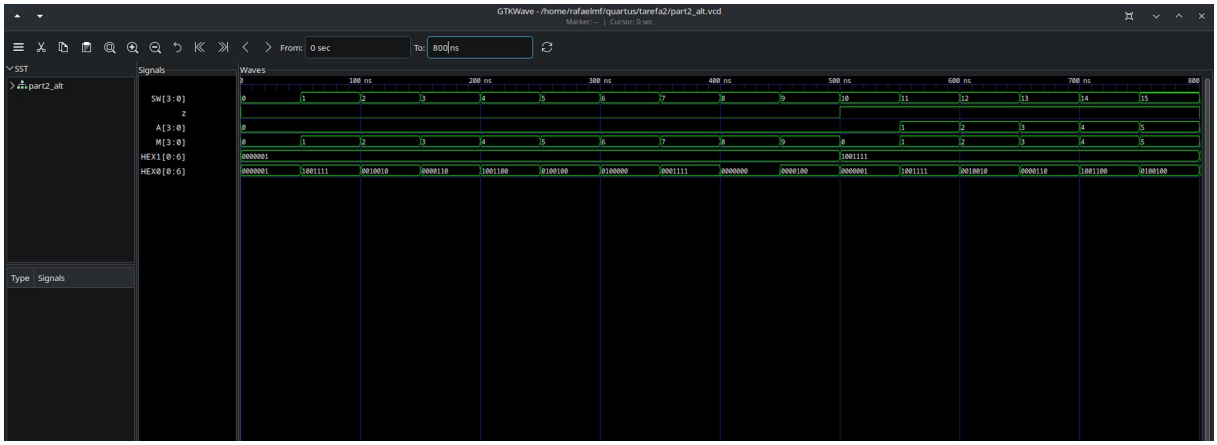


Figura 3 – Parte 2: V de 0 a 15

z sobe em V = 10. A partir daí, o circuito A gera 0 a 5, o multiplexador passa a entregar A em vez de V, e HEX1 muda para 1. Os displays mostram 00 a 15.

5.3 Parte 3

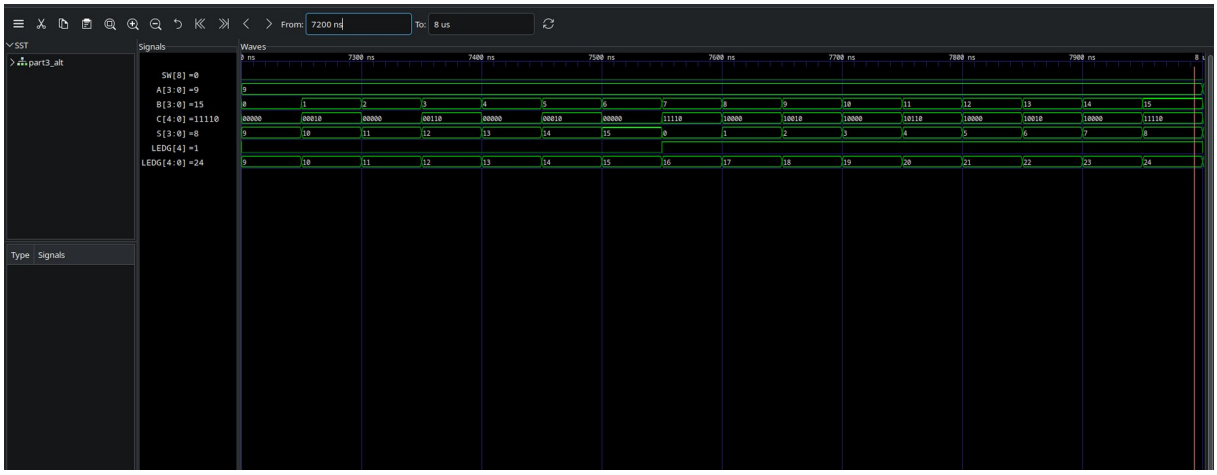


Figura 4 – Parte 3: A = 9, cin = 0, B de 0 a 15

Com A = 9, LEDG4–0 vai de 9 a 24. O carry de saída (LEDG4) sobe em B = 7, quando a soma passa de 15. A cadeia C(4..0) mostra o carry propagando por todos os bits (`11110`) nesse ponto.

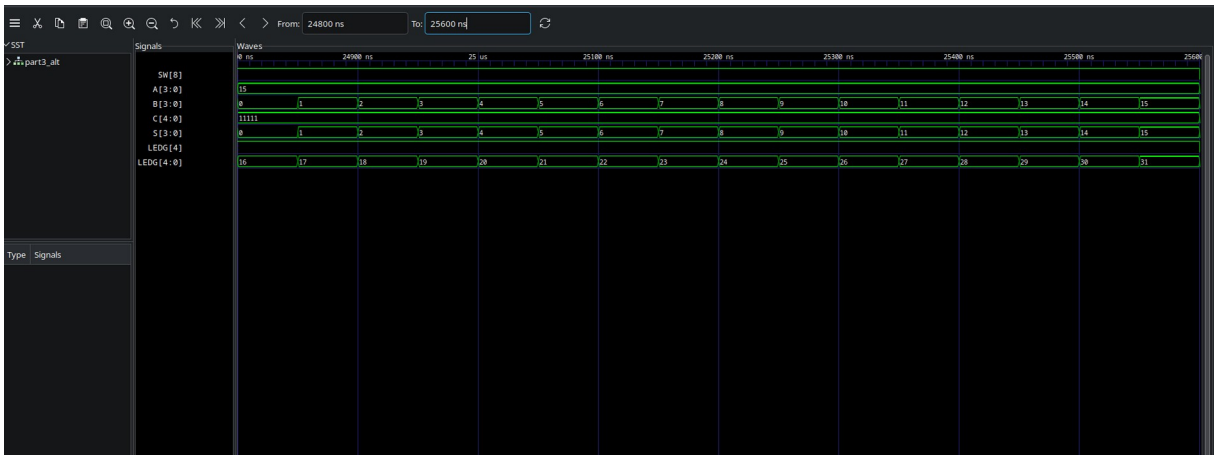


Figura 5 – Parte 3: A = 15, cin = 1, B de 0 a 15

Com A = 15 e cin = 1, a cadeia de carries fica toda em 11111 e a soma vai de 16 a 31. Isso confirma o caminho do carry de entrada.

5.4 Parte 4

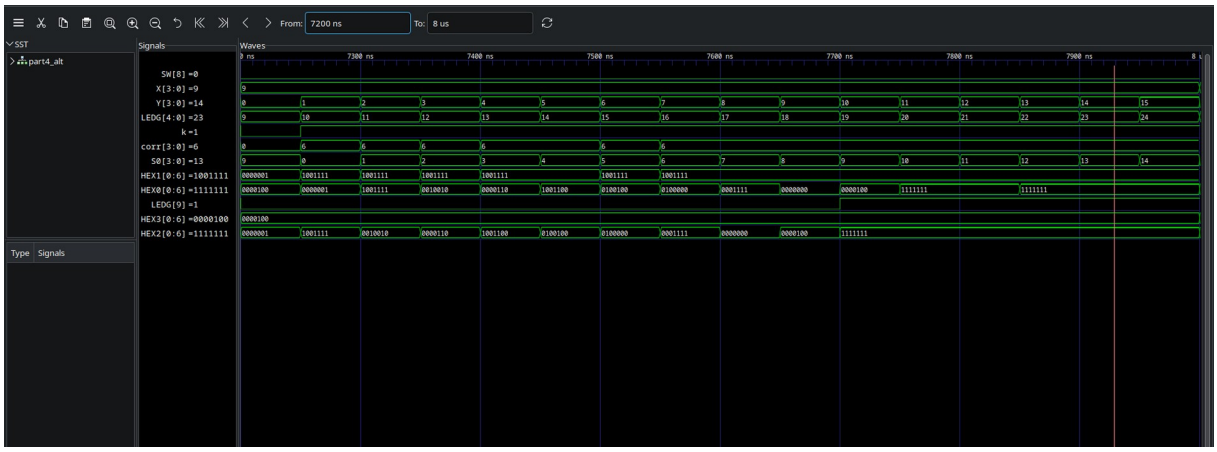


Figura 6 – Parte 4: X = 9, cin = 0, Y de 0 a 15

Para Y de 1 a 9, k = 1 e corr = 6, e S0 vira 0 a 8 com HEX1 = 1, ou seja, as somas 10 a 18. A partir de Y = 10, LEDG9 acende e HEX2 apaga, indicando entrada inválida.



Figura 7 – Parte 4: X = 9, cin = 1, Y de 0 a 15

Com cin = 1, as somas válidas vão de 10 a 19. Em Y = 9 aparece o maior resultado possível, 9 + 9 + 1 = 19.

5.5 Parte 5



Figura 8 – Parte 5: A = 9, cin = 0, B de 0 a 15

S1 = T / 10 e S0 = T MOD 10 acompanham a soma de 9 a 24. Para B > 9 o resultado continua correto (por exemplo, 9 + 11 = 20 mostra 20), embora a entrada seja inválida em BCD.

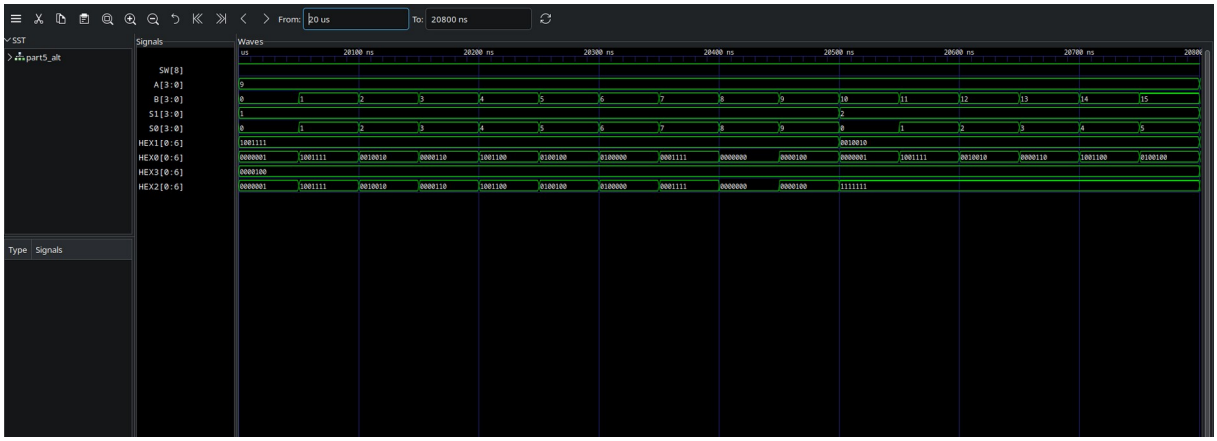


Figura 9 – Parte 5: A = 9, cin = 1, B de 0 a 15

Com cin = 1, as somas vão de 10 a 25. S1 passa de 1 para 2 em B = 10.

5.6 Parte 6



Figura 10 – Parte 6: SW de 0 a 31

A dezena muda em 10, 20 e 30. As correções de +3 aparecem nas células: em SW = 10, e7 = 5 vira c7 = 8, e o bit que transborda forma o 1 das dezenas. Em SW = 20, a correção acontece em e5 = 5 → c5 = 8.



Figura 11 – Parte 6: SW de 32 a 63

A partir de SW = 40 a primeira célula também corrige (e3 = 5 → c3 = 8). O maior valor, 63, é exibido como 63.

5.7 Comparação no RTL Viewer: Partes 4 e 5

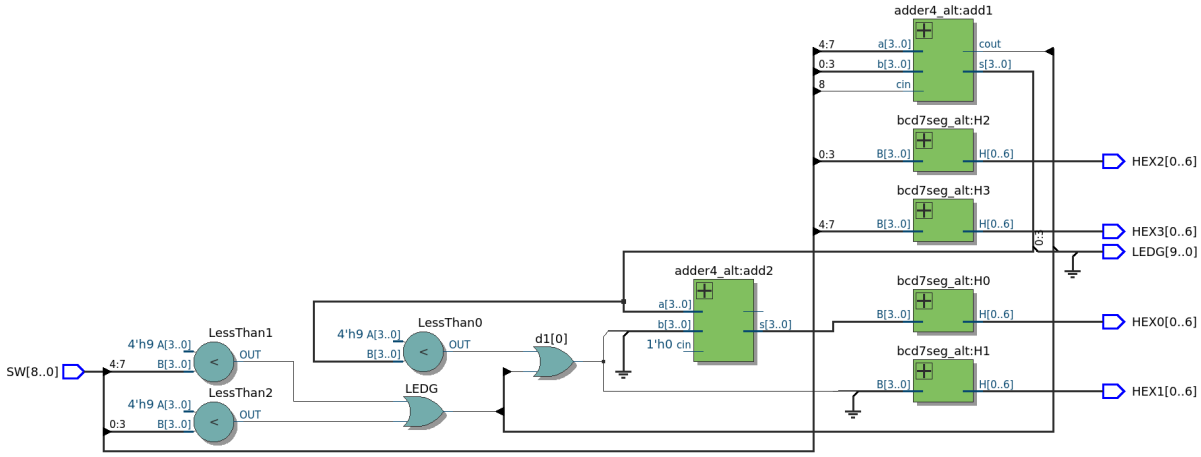


Figura 12 – RTL Viewer da Parte 4 (correção +6)

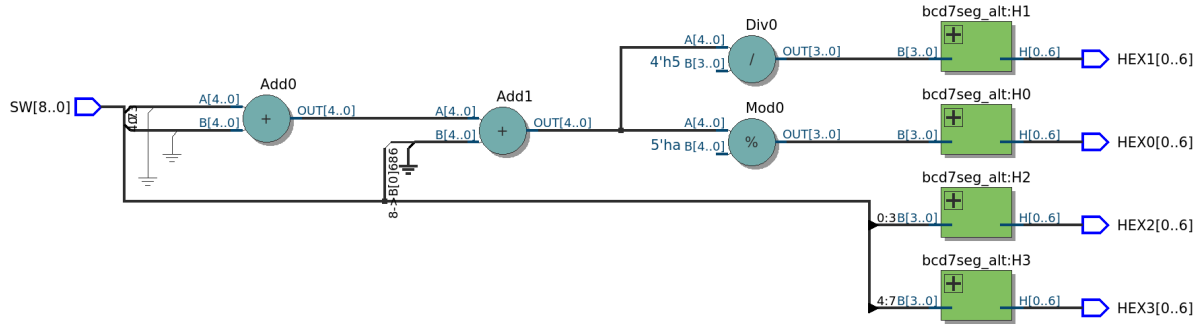


Figura 13 – RTL Viewer da Parte 5 (divisão e resto)

Aspecto	Parte 4 (estrutural)	Parte 5 (aritmética)
Soma	`adder4_alt:add1`, o ripple-carry descrito por nós	`Add0` e `Add1`, somadores inferidos do operador +
Decisão “> 9”	Comparador `LessThan0` e uma OR com cout geram k	Não existe
Correção	`adder4_alt:add2` soma `0110`; o `WHEN ELSE` de `corr` virou fios (k ligado aos bits 1 e 2)	`Div0` e `Mod0`, divisor e resto inferidos de `/` e `MOD`
Extras	`LessThan1` e `LessThan2` com uma OR geram LEDG9	—

Na Parte 4, o hardware gerado corresponde quase um a um ao que foi descrito. Na Parte 5, o código descreve só a matemática, e o sintetizador escolhe o circuito. Um exemplo é o `Div0`, que aparece com divisor 5 (`4'h5`) e não 10: aparentemente o Quartus reescreveu T / 10 como (T / 2) / 5, já que o bit menos significativo não afeta o quociente. O código

comportamental é mais curto e menos sujeito a erro de projeto, mas dá menos controle sobre a estrutura e sobre o custo do circuito gerado.

6. Conclusão

Os seis circuitos da Tarefa 2 foram implementados em duas versões cada. As seis soluções alternativas funcionaram corretamente em todas as combinações simuladas, 1.872 no total, sem nenhum erro.

O trabalho mostrou que o mesmo circuito combinacional pode ser descrito em VHDL em níveis de abstração bem diferentes:

- **Expressões booleanas (SOP).** Dão controle total sobre a lógica e exigem a etapa manual de tabela-verdade e mapa de Karnaugh. É nessa etapa que os don't cares das entradas BCD inválidas reduzem as equações.
- **Atribuições selecionadas e condicionais (`WITH SELECT`, `WHEN ELSE`).** Transcrevem a tabela-verdade diretamente e transferem a minimização para o sintetizador. O resultado é equivalente, com código mais legível e menos sujeito a erro.
- **Descrição estrutural com `FOR GENERATE`.** Torna o ripple-carry e o double dabble paramétricos e reaproveitáveis: o `fa_alt` da Parte 3 foi usado de novo no somador BCD da Parte 4.
- **Operadores aritméticos (`+`, `/`, `MOD`).** Produzem o código mais curto. O RTL Viewer mostrou, porém, que o sintetizador decide sozinho a estrutura, inclusive otimizando T / 10 para (T / 2) / 5.

As Partes 4 e 5 também mostraram que a mesma correção BCD pode ser feita de formas diferentes: subtraindo 10, somando 6 em módulo 16 ou usando divisão e resto. Na Parte 6, o double dabble substituiu a cadeia de comparações por faixas por apenas três células add3.

Por fim, a transcrição das soluções de referência mostrou limitações práticas. O Quartus aceita expressões no `PORT MAP`, mas o ModelSim em VHDL-93 não. Além disso, o roteiro foi escrito para placas com HEX5 e LEDR, que a DE0 não possui. Esses pontos foram resolvidos com sinais intermediários e com o remapeamento das saídas para HEX3/HEX2 e LEDG.

Referências

IMBIRIBA, R. **Laboratório de Sistemas Digitais Avançados – Exercício 02: Números e Display**. Fortaleza: Universidade de Fortaleza, 2026. Roteiro de laboratório (LABORATÓRIO SDA - TAREFA2.pdf).

IMBIRIBA, R. **Sistemas Digitais Avançados – Composição do relatório da tarefa 2**. Fortaleza: Universidade de Fortaleza, 2026. Orientações para o relatório (Entregáveis Relatório2.pdf).

IMBIRIBA, R. **Sistemas Digitais Avançados – Solução: Trabalho 2**. Fortaleza: Universidade de Fortaleza, 2026. Códigos de referência das Partes 1 a 6 (trabalho 2 - SDA - solução.pdf).